GPS-denied navigation of the Daedalus warehouse drone

System architecture, ROS 2 node structure and sensor-fusion design of the current configuration (`scripts/start.sh start --vio` + `/ws/src/fly_vio.sh`), simulated in PX4 SITL + Gazebo Harmonic, ROS 2 Humble.

> **In one paragraph.** The drone flies without GPS and without magnetometer. Its pose comes from a **tightly coupled stereo visual-inertial odometry** (OpenVINS, ZED 2i stereo pair + ZED IMU). The two quantities that any visual-inertial odometry cannot observe in absolute terms, **heading** and (in practice) **height**, are re-anchored to the warehouse itself: the orthogonal structure of racks and walls gives the heading (Manhattan world) and the floor seen in the depth image gives the height. The corrected pose is fused by the PX4 **EKF2** with the autopilot's IMU, barometer and a **multirotor drag model**; RTAB-Map builds the map on top of the EKF2 odometry and its loop closures correct the flight setpoints. On the full predefined path (268 m, 273 s) the absolute position error is 0.72 m RMS (1.17 m max) and the final height error 4 cm.

1. Sensors

Sensor	Simulation	Rate	Status	Used by
ZED 2i left colour image	Gazebo camera, VGA 672×376, HFOV 101° (f_x = 275 px) + ZED noise model (`camera_noise`)	15 Hz	**on**	OpenVINS, RTAB-Map
ZED 2i right image	not rendered: synthesized from left image + depth (`stereo_synth`), baseline 0.12 m, independent noise	15 Hz	**on**	OpenVINS
ZED 2i depth (ZED SDK output)	Gazebo depth camera, pixel-aligned with the left image + ZED depth noise model	15 Hz	**on**	manhattan_yaw, RTAB-Map
ZED 2i IMU (Bosch BMI085)	Gazebo IMU on the camera body; datasheet white noise, assumed bias random walk	250 Hz (real: 400 Hz; capped by the 4 ms physics step)	**on**	OpenVINS
PX4 IMU	PX4 SITL sensor on `base_link`	250 Hz	**on**	EKF2 prediction and drag model
Barometer	PX4 SITL	–	**on**	EKF2, secondary height aid
GPS	PX4 SITL	–	**off**	– (`SYS_HAS_GPS=0`, `EKF2_GPS_CTRL=0`)
Magnetometer	PX4 SITL	–	**off**	– (`SYS_HAS_MAG=0`, `EKF2_MAG_TYPE=5`): steel racks disturb it; it caused a 2.7° heading error

Prior knowledge used (not sensors): the **dock heading** the drone takes off from (only needed within ±45°), the **orientation of the warehouse axes** (direction of the racks) and the **occupancy map** for path planning. The Gazebo ground truth is used only by the evaluation scripts, never in flight.

2. Fusion architecture at a glance

The design is **layered**: each layer runs at its own rate and corrects what the layer below cannot observe. Fast layers are tightly coupled (raw measurements in one estimator); slow absolute references enter as gated, rate-limited corrections of the frame alignment, so they never make the estimate jump.

Layer	Component	Inputs	Rate / time scale	What it provides	What it cannot provide
1. Visual-inertial odometry	OpenVINS (MSCKF, tightly coupled)	stereo pair, ZED IMU	IMU 250 Hz, images 15 Hz	metric pose, velocity, roll/pitch (gravity), IMU biases	absolute yaw and position (they drift: up to 8° in a flight)
2. Absolute references from the structure	manhattan_yaw + vision_to_px4	depth, roll/pitch from layer 1	15 Hz measurements, filter τ = 3 s (yaw), 2 s (height)	absolute heading (mod 90°), height above floor	horizontal position
3. Autopilot state estimator	PX4 EKF2	vision pose/velocity/yaw, PX4 IMU, baro, drag	IMU 250 Hz, vision 15 Hz	smooth, low-latency state for control; bridges short vision gaps (drag model)	does not reduce drift by itself
4. Global consistency	RTAB-Map + offboard_control	colour, depth, cloud, EKF2 odometry	new node every 10 cm / ~6°; loop closures sporadic	map, loop-closure correction `map→odom` applied to the setpoints	–

Which source constrains which state

State	ZED IMU	Stereo images	Manhattan (depth)	Floor (depth)	PX4 IMU	Baro	Drag model	Loop closures
Roll, pitch	**absolute (gravity)**	relative	–	–	**absolute**	–	–	–
Yaw	short term	relative (drifts)	**absolute, mod 90°**	–	short term	–	–	map only
Horizontal position	short term	relative (drifts)	–	–	short term	–	–	setpoints / map
Height	short term	relative (drifts)	–	**absolute**	short term	absolute, drifts	–	–
Horizontal velocity	short term	**metric (stereo)**	–	–	short term	–	**without vision**	–
IMU biases	**estimated by OpenVINS**	**(observable)**	–	–	**estimated by EKF2**	–	–	–

Two different IMUs are used on purpose: the ZED IMU inside OpenVINS, the PX4 IMU inside EKF2. Re-using the same IMU in both estimators would count its information twice and make EKF2 over-confident.

3. ROS 2 node graph

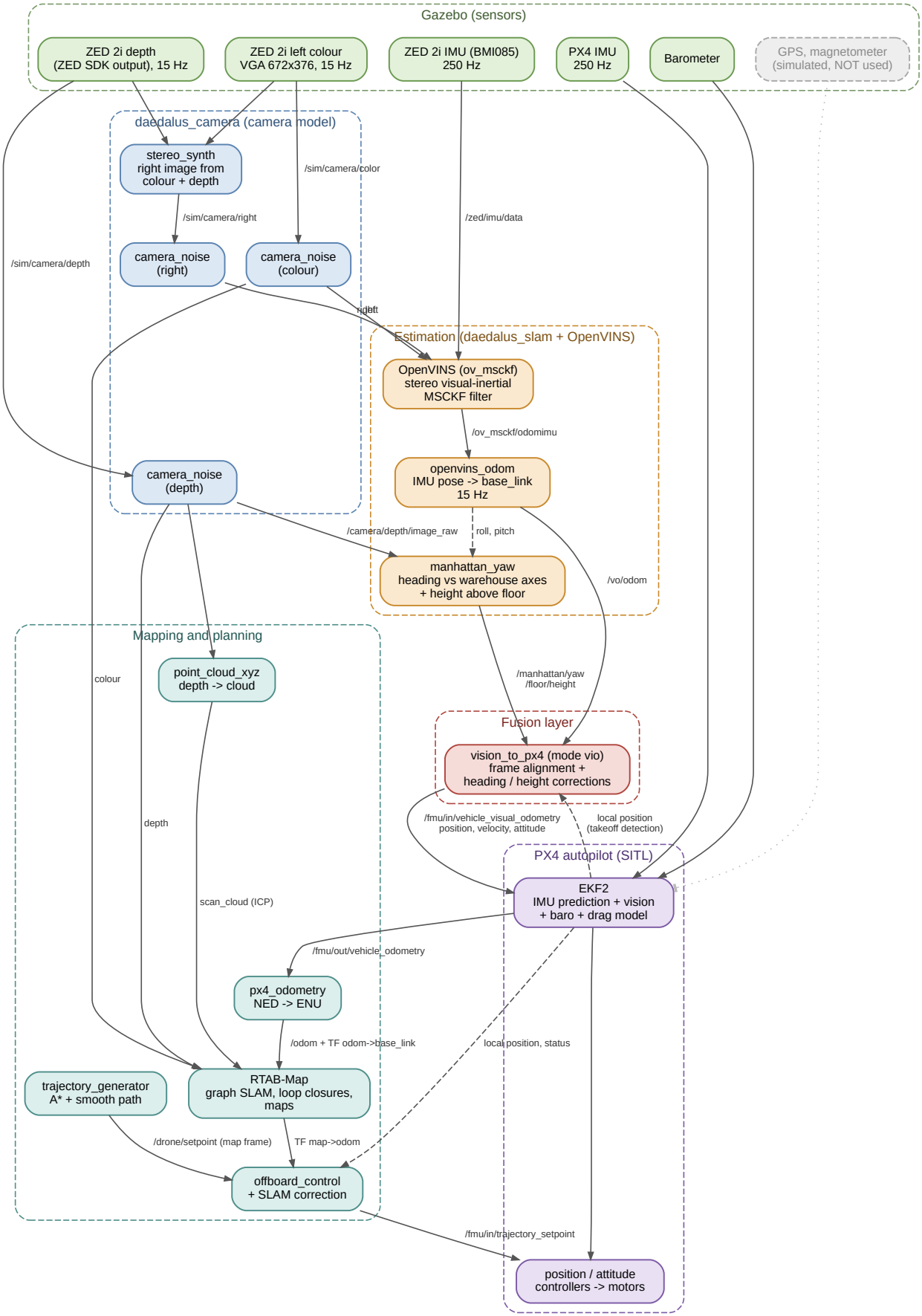


Figure 1 – Nodes and topics of the configuration `odom:=openvins`. Green: simulated sensors (Gazebo, bridged by `ros_gz_bridge`, not drawn); blue: camera model; orange: estimation; red: fusion layer; violet: PX4 (SITL, connected through the micro-XRCE-DDS agent); teal: mapping, planning and offboard control. Dashed edges: auxiliary data.

4. Nodes

Node (package)	Subscribes	Publishes	Role
`parameter_bridge` (ros_gz_bridge)	Gazebo topics	`/clock`, `/sim/camera/{color,depth}/image_raw`, camera_info, `/imu/data`, `/zed/imu/data`	Gazebo → ROS. Images go to `/sim/...` (ideal), the noisy ones are on `/camera/...`
`camera_noise` ×3 (daedalus_camera)	ideal colour / depth / right	`/camera/{color,depth,right}/image_raw`	ZED 2i noise model (read + shot noise; depth error growing with range, holes, edges), independent seeds
`stereo_synth` (daedalus_camera)	ideal colour + depth + camera_info	`/sim/camera/right/image_raw`, `/camera/right/camera_info`	right image by depth-image-based rendering: each pixel shifted by $f_x b/Z$, z-buffer, holes filled with the background. Checked with OpenCV SGBM: median disparity error 0.24 px
`run_subscribe_msckf` (ov_msckf, OpenVINS 2.7)	left, right, `/zed/imu/data`	`/ov_msckf/odomimu` (IMU pose, 250 Hz)	stereo visual-inertial MSCKF (section 5.1)
`openvins_odom` (daedalus_slam)	`/ov_msckf/odomimu`, TF	`/vo/odom` (15 Hz)	IMU pose → `base_link` pose (lever arm, covariance propagated), origin at the start
`manhattan_yaw` (daedalus_slam)	`/camera/depth/image_raw`, camera_info, `/vo/odom`, TF	`/manhattan/yaw`, `/floor/height`	heading relative to the warehouse axes and height above the floor, from surface normals (5.2, 5.3)
`vision_to_px4` (daedalus_slam, mode vio)	`/vo/odom`, `/manhattan/yaw`, `/floor/height`, PX4 local position	`/fmu/in/vehicle_visual_odometry`, diagnostics `/manhattan/correction`, `/floor/correction`	aligns the visual frame to the warehouse, applies the heading and height corrections, sends pose + velocity + attitude with covariance to PX4 (5.4)
EKF2 (PX4)	vision, IMU, baro	`/fmu/out/vehicle_odometry`, `vehicle_local_position`	autopilot state estimator (5.5)
`px4_odometry` (daedalus_slam)	`/fmu/out/vehicle_odometry`	`/odom`, TF `odom→base_link`	NED/FRD → ENU/FLU
`point_cloud_xyz` (rtabmap_util)	noisy depth	`/rtabmap/scan_cloud`	point cloud (decimated, voxel 5 cm, ≤ 5 m) for ICP
`rtabmap` (rtabmap_slam)	colour, depth, scan_cloud, `/odom`	TF `map→odom`, 3D cloud map, 2D occupancy grid	graph SLAM with visual + ICP loop closures, robust optimizer
`trajectory_generator` (daedalus_trajectories)	– (known occupancy map)	`/drone/setpoint`	A* through the targets, smooth trajectory (map frame)
`offboard_control` (daedalus_trajectories)	`/drone/setpoint`, TF `map→odom`, PX4 status/position	`/fmu/in/trajectory_setpoint`, mode, commands	arms, offboard mode; converts map-frame setpoints into PX4's frame with the SLAM correction, applied gradually

5. Sensor-fusion design

5.1 Layer 1 – tightly coupled stereo visual-inertial odometry (OpenVINS)

OpenVINS is a Multi-State Constraint Kalman Filter (MSCKF). Its state holds the IMU pose, velocity and gyroscope / accelerometer biases, a sliding window of 11 past camera poses (clones) and up to 50 long-lived landmarks. The IMU is integrated at 250 Hz (RK4); every stereo frame (15 Hz, 200 KLT features per camera) updates the state with the reprojection errors of the tracked features. Because raw feature measurements and IMU are fused in one filter (*tight coupling*), the IMU predicts where features move (robust in turns and blur), the stereo baseline fixes the metric scale and the biases are estimated online.

- **Calibration:** camera–IMU extrinsics from the static TFs; online refinement of extrinsics and time offset **disabled**: in a first test it stretched the stereo baseline and produced a 4% scale error.
- **IMU noise:** BMI085 datasheet densities ×2.5 (gyro $6.1 \cdot 10^{-4}$ rad/s/√Hz, accel $2.95 \cdot 10^{-3}$ m/s^2/√Hz).
- **Initialization:** static, on the ground; with zero-velocity updates enabled it does not wait for a takeoff jerk, so PX4 receives vision before arming.
- **Output:** `openvins_odom` converts the IMU pose into the `base_link` pose ($p_B = p_I + R_{GI}\, r$, covariance through the Jacobian of the lever arm) and decimates it to 15 Hz.

Measured on a recorded flight (replay): relative error 1.39% over 10 m, 0 tracking resets over 265 m, 0.4° of yaw error added per 90° turn (RGB-D odometry previously used: 2.1%, 4 resets, 2.2° per turn). What remains is the slow drift of the unobservable directions: yaw (up to 8° in a flight) and height (up to 1.2 m).

5.2 Layer 2a – absolute heading from the warehouse structure (Manhattan world)

Racks, uprights and walls lie along two orthogonal directions that are known in the map. On every depth image `manhattan_yaw` computes surface normals on a 4-px grid (cross product of the 3D neighbours; points closer than 5 m; depth edges rejected with the second difference of the inverse depth, which is zero on any plane, also seen at grazing angle). The normals are rotated to `base_link` and levelled with the roll and pitch of OpenVINS (observable through gravity); only near-horizontal ones (vertical surfaces) are kept. Their angles φ_i are four-fold symmetric, so:

$$\theta = \tfrac{1}{4}\,\mathrm{atan2}\Big(\textstyle\sum w_i \sin 4\varphi_i\,,\ \sum w_i \cos 4\varphi_i\Big), \quad \text{quality } q = \Big|\textstyle\sum w_i\, e^{4j\varphi_i}\Big| \,/\, \sum w_i \in [0,1]$$

$$\text{heading measurement: } \chi_{meas} \equiv \beta + \theta \ (\mathrm{mod}\ 90°),\ \beta = \text{heading of the warehouse axes (0° in the simulation)}$$

`vision_to_px4` compares it with the current heading estimate χ_{est} at the image timestamp (history of the odometry, extrapolated up to 150 ms) and corrects the frame alignment:

$$e = \mathrm{wrap}_{[-45°,\,45°)}\big(\chi_{meas} - \chi_{est}\big)$$

$$\text{accepted if } q \geq 0.45,\ \geq 500 \text{ normals},\ |\text{yaw rate}| < 10°/s,\ |e| < 5°\ (45° \text{ on the ground, first lock})$$

$$\Delta\psi = \mathrm{clamp}\big(e \cdot \Delta t / \tau,\ \pm 0.3°/s \cdot \Delta t\big),\ \ \tau = 3\ s;\quad R_{align} \leftarrow R_z(\Delta\psi)\, R_{align},\ \ o \leftarrow p_{last} - R_{align}\, p_{vo,last}$$

The rotation is applied about the **current position** (offset recomputed), so the position sent to PX4 never jumps: only the motion from then on is rotated. The ambiguity of 90° is resolved by the closest branch, which is why the dock heading is only needed within ±45°. Accuracy per image: median 0.95°, RMS 1.6°, bias +0.4°, usable on 89% of the images; in the closed loop the heading sent to PX4 has an RMS error of about 1.3° (OpenVINS alone: up to 8°).

5.3 Layer 2b – absolute height from the floor

In the same pass, normals within 10° of the vertical belong to horizontal surfaces. Their points below `base_link` (up to 7 m: the forward camera sees the floor only from about 4.4 m ahead at 3 m of altitude) give candidate heights. Box tops and pallets are horizontal too, but higher: the **floor is the lowest surface with enough points** (2 cm histogram, lowest bin with ≥ 40 points with its neighbours, refined by the mean within ±4 cm).

floor level in the map frame: z_{floor} = median($h_{est} - h_{meas}$) over the first 20 measurements after takeoff
$e_h = z_{floor} + h_{meas} - h_{est}$, accepted if $|e_h| < 0.3$ m; $o_z \leftarrow o_z - \text{clamp}(e_h \, \Delta t / \tau, \pm 5 \text{ cm/s} \cdot \Delta t)$, $\tau = 2$ s

The reference is taken in flight because `base_link` is at the feet: on the ground no floor is below it. A constant bias of the measurement (+7 cm) cancels, since the reference carries the same bias. Accuracy: RMS 0.1 m per image at 15 Hz; height sent to PX4 within 8 cm RMS over a whole flight.

5.4 Frame alignment (vision_to_px4)

OpenVINS works in its own frame `vo_odom`: gravity-aligned, origin at the start, arbitrary heading. The node maps it to PX4's local NED frame (= warehouse frame, origin at the dock) with a rotation about the vertical and an offset:

$$p_{NED} = R_z(\psi_a) \, p_{vo} + o, \quad R_{NED} = R_z(\psi_a) \, R_{vo}, \quad \text{initially } \psi_a = \chi_{dock} - \chi_{vo}(0), \; o = -R_z(\psi_a) \, p_{vo}(0)$$

ψ_a is then corrected by 5.2 and o_z by 5.3. Each message carries position, body-frame velocity and attitude with OpenVINS' variances (horizontal variances rotated conservatively: the larger of x and y). A `dropouts` parameter can suspend the messages to test EKF2 without vision.

5.5 Layer 3 – PX4 EKF2

EKF2 predicts with the PX4 IMU and fuses the external vision (position, velocity and yaw), the barometer and the multirotor drag model. The drag model exploits that the thrust of a multirotor is along body z: the horizontal accelerometer reading is pure aerodynamic drag, a = −MCOEF · v (body frame), which observes the horizontal velocity also when vision is missing. MCOEF was fitted on a recorded flight (`fit_drag.py`): purely linear drag, 0.115 s^{-1}, residual 0.003 m/s^2; no quadratic (bluff-body) term.

Parameter (set by `start.sh --vio`)	Value	Meaning
`SYS_HAS_GPS` , `EKF2_GPS_CTRL`	0, 0	no GPS
`SYS_HAS_MAG` , `EKF2_MAG_TYPE`	0, 5	no magnetometer fusion
`EKF2_EV_CTRL`	15	vision horizontal position + vertical position + 3D velocity + yaw
`EKF2_HGT_REF`	3	height reference: vision (barometer still fused)
`EKF2_EV_NOISE_MD` / `EVP` , `EVV` , `EVA_NOISE`	0 / 0.05 m, 0.05 m/s, 0.05 rad	OpenVINS' variances, lower-bounded by these values
`EKF2_EV_DELAY`	5 ms	OpenVINS output is propagated to the latest IMU sample (measured ≈ 0 ms)
`EKF2_DRAG_CTRL` , `EKF2_MCOEF` , `EKF2_BCOEF_X/Y`	1, 0.115, 0	drag fusion, linear propeller drag only

5.6 Layer 4 – SLAM and setpoint correction

RTAB-Map uses the EKF2 odometry (`/odom`) as motion prior and adds a graph node every 10 cm or ~6°. Loop closures are found visually and refined with ICP on the depth cloud; a robust optimizer (GTSAM + Vertigo) down-weights inconsistent ones. The result is the TF `map→odom` . `offboard_control` receives the setpoints in the map frame and converts them into PX4's frame with this correction, applied gradually so a loop closure never makes the setpoint jump. In the current flights loop closures change the trajectory very little (final graph 0.72 m RMS, same as the odometry): this is the next improvement.

6. Frames

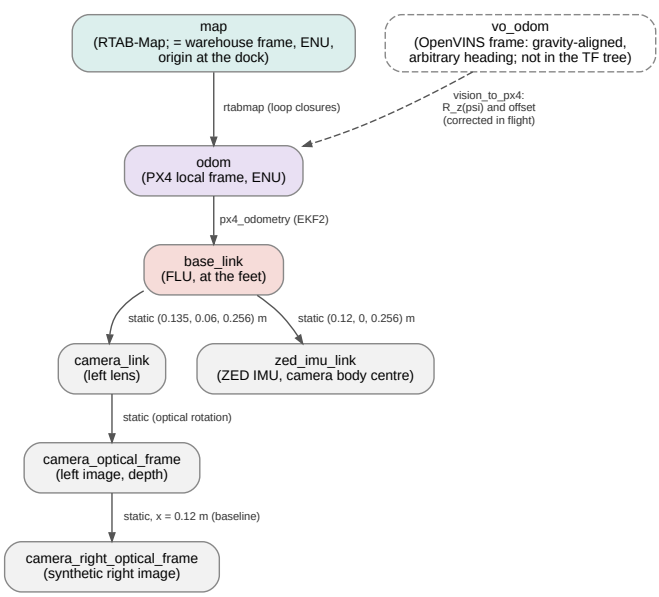


Figure 2 – TF tree. `vo_odom` is not in the tree: it exists only inside `vision_to_px4`, whose alignment links it to PX4's frame.

Conventions. ROS frames are ENU (x east, y north, z up) and FLU for bodies; PX4 uses NED / FRD. Conversions happen only in `vision_to_px4` (to PX4) and `px4_odometry` (from PX4).

map = warehouse frame. The planner's targets are expressed with the origin at the dock and the axes of the world; without GPS this frame is realized by the dock heading and the Manhattan correction (orientation) and by the take-off point (origin).

Rates. IMU 250 Hz (both), images and depth 15 Hz, `/vo/odom` and the vision messages to PX4 15 Hz, Manhattan and floor measurements 15 Hz, offboard setpoints 10 Hz.

7. Simulation specifics

- **Synthetic stereo.** Rendering a second camera would add GPU load; the right image is derived from the left image and the ideal depth. Limits: same exposure and lighting as the left image, no view-dependent effects, filled strips behind edges; disparity is zero beyond the 20 m clip of the depth camera.
- **Noise.** Colour (read + shot noise), depth (range-dependent error, spatially correlated, holes and edge errors, fitted to published ZED 2i accuracy data), BMI085 IMU (datasheet densities; bias random walk assumed, to be measured with an Allan variance on the real camera).
- **Optimistic aspects.** Perfect calibration and synchronization, no rolling shutter, no vibrations: on the real drone the numbers will be worse, and the IMU noise and the gates may need re-tuning.

8. Results (no-GPS flights, full predefined path, closed loop)

Configuration	Result	Abs. error RMSE	Abs. error max	Heading offset (mean)	Final height error
Initial: RGB-D odometry + magnetometer	crash at 66 s / 77 m	1.74 m	3.0 m	−2.4°	–
OpenVINS, no magnetometer, drag	crash at 113 s / 117 m	1.05 m	2.6 m	−3.5° (peak −7°)	–
+ Manhattan heading	path completed, 267 m	0.98 m	2.08 m	−0.5°	−0.83 m
+ floor height (current)	**path completed, 268 m**	**0.72 m**	**1.17 m**	**+0.3°**	**+0.04 m**

Absolute error = position estimate used by the drone vs Gazebo ground truth, without any alignment (the error the drone flies with). One flight per configuration. Plots and metrics: `images/odom_eval/<run>/`.

Open points: loop closures barely improve the map yet; residual ±2–3° heading excursions of the Manhattan measurement where the scene is not orthogonal; the vision-dropout test of the drag model is still to be run.

9. How to run

```
# host: PX4 + Gazebo without GPS and magnetometer, opens the ROS 2 shell
scripts/start.sh start --vio

# in the shell (dev container): build + predefined path with the full pipeline
/ws/src/fly_vio.sh rviz:=true
#   comparison switches: manhattan:=false   floor:=false

# evaluation against ground truth (plots and metrics in images/odom_eval/<name>)
/ws/src/daedalus_slam/scripts/odom_eval/run_eval.sh <name> odom:=openvins

# host: stop everything
scripts/start.sh stop
```